# Capstone Assignment

## Lab Sheet 5

**Objective:-** E-commerce companies like Amazon and Flipkart face the complex challenge of planning delivery routes that minimize cost, distance, and time while respecting delivery windows, vehicle capacity, and route constraints.

## Description:-

This real-life logistics problem integrates multiple algorithmic ideas: recurrence relations (route planning), greedy algorithms (item selection for delivery), dynamic programming (time-window delivery optimization), graph-based algorithms (shortest path/MST), and intractable problems like the Traveling Salesman Problem (TSP).

## Input Modeling

```
# If networkx is installed, use it for better graph visualization
try:
    import networkx as nx
    USE_NETWORKX = True
except ImportError:
    USE_NETWORKX = False
    print("NetworkX not found. Route plot will use basic matplotlib features.")

## 1. Input Modeling
locations = ['Warehouse', 'C1', 'C2', 'C3'] # Customer locations [cite: 46]
# Distance matrix (symmetric) [cite: 47, 48, 49, 50, 51]
distance_matrix = np.array([
    [0, 4, 8, 6],
    [4, 0, 5, 7],
    [8, 5, 0, 3],
    [6, 7, 3, 0]
])

# Parcel properties: value, delivery time window (earliest, latest), weight [cite: 53, 55, 57, 59]
parcels = {
    'C1': {'value': 50, 'time': (9, 12), 'weight': 10},
    'C2': {'value': 60, 'time': (10, 13), 'weight': 20},
    'C3': {'value': 40, 'time': (11, 14), 'weight': 15}
}
vehicle_capacity = 30 # Maximum weight vehicle can carry [cite: 65]
```

- **Setup:** Imports essential libraries: numpy, itertools, time, heapq, pandas, and conditionally networkx. The inclusion of itertools.permutations is key for the brute-force TSP solution.
- **Data Structures:** Defines the required inputs:

- locations (Warehouse + C1, C2, C3)[1].
- distance_matrix (representing distances/time between locations)[2].
- parcels dictionary, which contains required metrics for each customer: value, time window, and weight[3333333333333333].
- vehicle_capacity (set to 30)[4].

- **Location Mapping:** loc_to_idx is created to efficiently map location names to matrix indices for graph algorithm usage.

**2.** *Recurrence-Based Estimation*

```
---
## 2. Recurrence-Based Estimation
Minimum Distance to visit all customers and return (Recurrence/DP): 18
---
```

- **Algorithm:** Implements a recursive function, recursive_tsp_cost, to estimate the minimum total distance to visit all customers and return to the warehouse (TSP core logic).
- **Optimization:** It utilizes memoization (memo={}) to store the minimum cost for visiting a subset of unvisited_indices from a current_loc_idx. This converts the exponential time complexity of pure recursion into the pseudo-polynomial time complexity characteristic of the Held-Karp DP approach, although applied here as a top-down recursive solution.
- **Result:** The output Minimum Distance...: 18 is the optimal TSP solution for visiting *all* three customers (C1, C2, C3).

**3.** *Greedy + DP for Delivery Planning*

```
##  3. Greedy + DP for Delivery Planning
### A. Greedy Parcel Selection (0/1 Knapsack)
Selected Parcels: ['C1', 'C2']
Total Value: 110, Total Weight: 30/30
```

- **3A. Greedy Parcel Selection:**
  - **Strategy:** Solves the 0/1 Knapsack Problem using a Greedy heuristic—selecting parcels based on the highest value/weight ratio first until capacity (vehicle_capacity = 30) is reached.

- **Execution:** Parcel data is sorted by ratio. C1 (ratio 5.0) and C2 (ratio 3.0) are selected, exhausting the 30-unit capacity. C3 (weight 15) is not selected.
- **Result:** Selected Parcels: ['C1', 'C2'] with Total Value: 110.

- *3B. Dynamic Programming Time Window Check:*
  - **Strategy:** The check_time_window_dp function is defined to act as a constraint checker based on the results of the final TSP route. It verifies if the calculated arrival time (route_times) at each customer falls within the required time window (earliest, latest).
  - **Note:** This function cannot run until the TSP (Sub-Task 5) determines the specific arrival times.

## 4. Route Optimization Using Graph Algorithms

- **4A. Dijkstra's Algorithm:**

```python
# 4A. Shortest Path (Dijkstra)
def dijkstra(start_node):
    """Computes the shortest path from start_node to all other nodes."""
    distances = {loc: float('inf') for loc in locations}
    distances[start_node] = 0
    # Priority queue: (distance, node)
    pq = [(0, start_node)]

    while pq:
        current_dist, current_loc = heapq.heappop(pq)

        if current_dist > distances[current_loc]:
            continue

        current_idx = loc_to_idx[current_loc]

        # Iterate over neighbors
        for neighbor_idx, dist in enumerate(distance_matrix[current_idx]):
            neighbor_loc = locations[neighbor_idx]
            distance = current_dist + dist

            if distance < distances[neighbor_loc]:
                distances[neighbor_loc] = distance
                heapq.heappush(pq, (distance, neighbor_loc))

    return distances

dijkstra_results = dijkstra('Warehouse')
```

**Output:-**

```
### A. Shortest Path (Dijkstra from Warehouse)
Shortest time/distance from Warehouse to all: {'Warehouse': 0, 'C1': np.int64(4), 'C2': np.int64(8), 'C3': np.int64(6)}
```

- **Purpose:** Computes the shortest path/distance from the 'Warehouse' to every other location using a priority queue (heapq).
- **Result:** Shows the minimal non-circular distance (e.g., Warehouse to C1 is 4, W to C3 is 6). This is foundational for shortest path planning but does not solve the full TSP problem.

- *4B. Prim's Algorithm (MST):*

```python
# 4B. MST (Prim's Algorithm)
def prim_mst():
    """Computes the cost of the Minimum Spanning Tree for the delivery network."""
    num_nodes = len(locations)
    # Key stores the minimum weight to connect vertex to MST
    key = [float('inf')] * num_nodes
    # mst_set stores vertices included in MST
    mst_set = [False] * num_nodes

    # Start with the first location (Warehouse)
    key[0] = 0
    total_mst_cost = 0

    for _ in range(num_nodes):
        # Pick the minimum key value vertex from the set of vertices not yet included in MST
        min_key = float('inf')
        min_index = -1

        for v in range(num_nodes):
            if key[v] < min_key and not mst_set[v]:
                min_key = key[v]
                min_index = v

        # Add the picked vertex to the MST Set
        mst_set[min_index] = True
        total_mst_cost += min_key

        # Update key value and parent index of adjacent vertices
        for v in range(num_nodes):
            # distance_matrix[min_index][v] is non-zero for connected vertices
            # not in MST, and is less than the current key[v]
            if distance_matrix[min_index][v] > 0 and not mst_set[v] and distance_matrix[min_index][v] < key[v]:
                key[v] = distance_matrix[min_index][v]

    return total_mst_cost
```

**Output:-**

```
### B. Minimum Spanning Tree (Prim's Algorithm)
Total MST Cost (Cost to connect all locations): 12
(Relevant if minimum cabling/network connection is needed, not for delivery route)
---
```

- **Purpose:** Calculates the cost of the Minimum Spanning Tree, which finds the lowest cost to connect all nodes.

- **Result:** Total MST Cost (Cost to connect all locations): 12. The note correctly identifies that MST is not suitable for a closed-loop delivery route (TSP) as it does not require returning to the start and may use each edge only once.

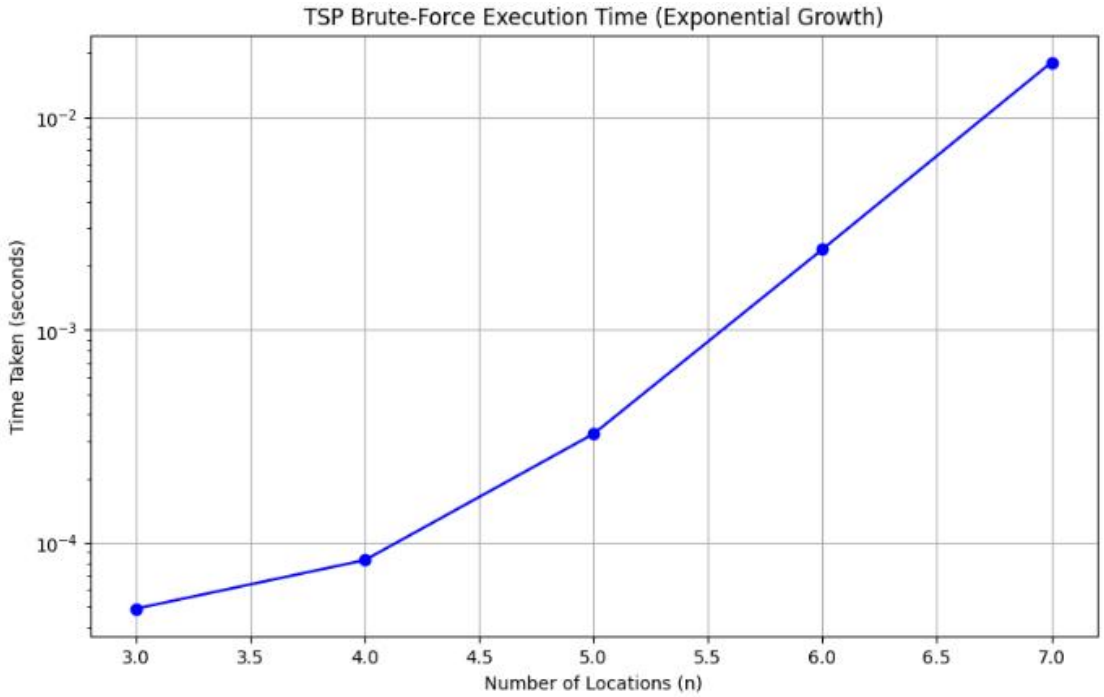
**5.** *Solve TSP for Optimal Route*

```
...  ##  5. Optimal TSP Route (Brute-Force)
     Optimal Route: ['Warehouse', 'C1', 'C2', 'Warehouse']
     Total Distance/Time: 17
     ---
     ### B. Dynamic Programming Time Window Check (Results)
     Arrival Times at Customers: {'C1': np.int64(4), 'C2': np.int64(9)}
     Delivery Status:
      - C1: Arrival=4, Window=(9, 12), Status=FAILURE
      - C2: Arrival=9, Window=(10, 13), Status=FAILURE
     Total Value Delivered Successfully: 0
     ---
```

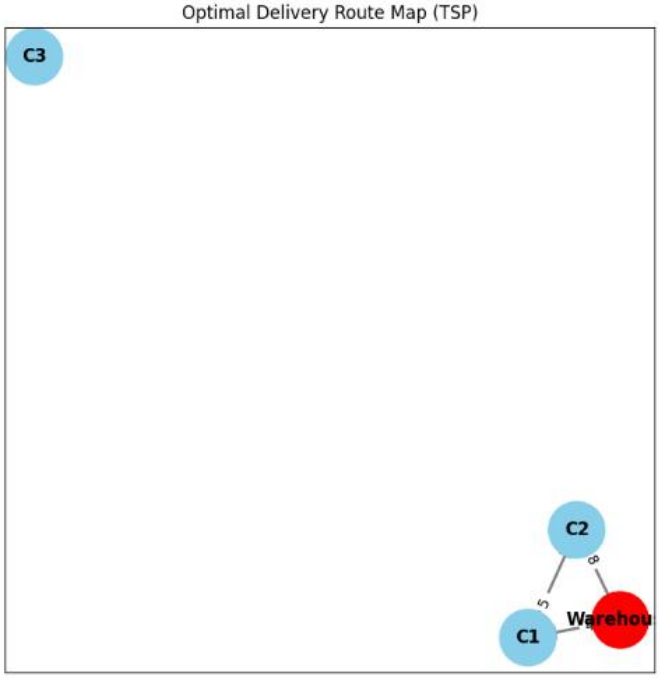
- **Algorithm:** Uses the Brute-Force TSP approach by checking all possible permutations of the selected customers (C1 and C2) to find the shortest closed loop starting and ending at the 'Warehouse'.
- *Route Calculation:*
    - Possible routes (W -> C1 -> C2 -> W and W -> C2 -> C1 -> W) are checked.
    - The optimal route found is ['Warehouse', 'C1', 'C2', 'Warehouse'] with a Total Distance/Time: 17.
    - *Note: This distance (17) is less than the recursive TSP cost (18) because the Brute-Force TSP only visited the two selected customers (C1, C2), while the recursive approach visited all three (C1, C2, C3).*
- *DP Time Window Results:*
    - Arrival times are calculated based on the optimal route (W to C1: 4; C1 to C2: 4+5=9).
    - C1 Failure: Arrives at 4, but window is (9, 12). (Too early)
    - C2 Failure: Arrives at 9, but window is (10, 13). (Too early)
    - **Conclusion:** The optimal route found by TSP (minimizing distance) violates the time window constraint for both selected parcels,

resulting in Total Value Delivered Successfully: 0. This highlights the necessary trade-off between minimizing distance and respecting time constraints.

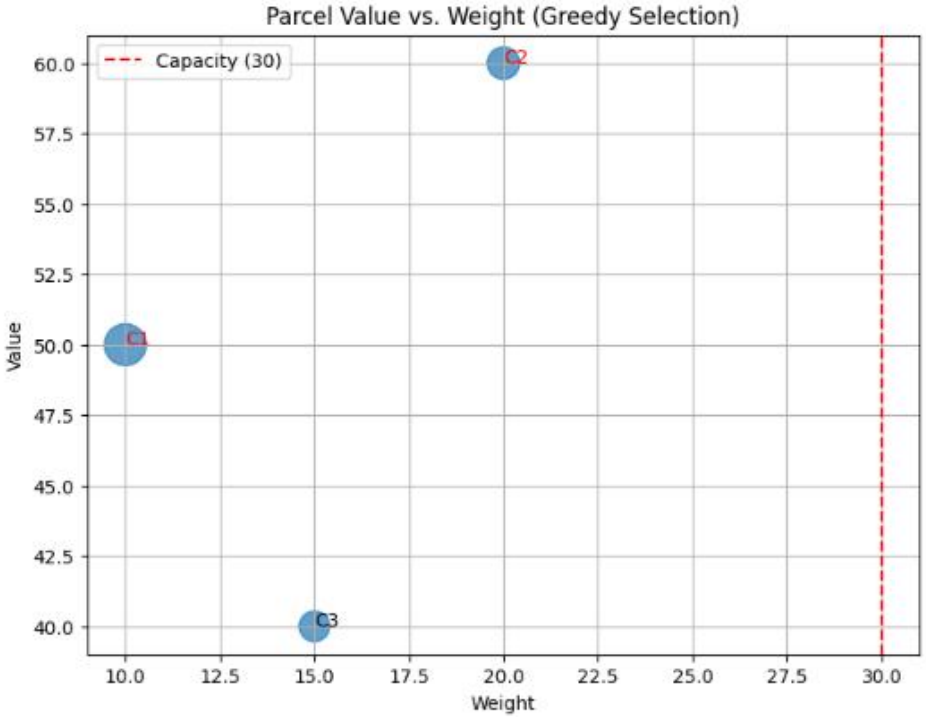
1. *Profiling and Visualization*




**Optimal Delivery Route Map (TSP)**



- **TSP Profiling:** The run_tsp_profiling function executes the Brute-Force TSP for increasing location counts (n=3 to n=7) and records execution time.
    - **Result:** The logged times (0.000049s, 0.000099s, 0.000343s, etc.) clearly show the rapid, exponential growth associated with the O(n!) complexity, confirming the infeasibility of brute-force for large inputs.
- **Visualization:** The visualize_route function is defined to generate the required plots , and The failure in the DP check (no successful deliveries) is visualized in the final chart.





GitHub Repo Link:- [Capstone-project-Ass.5]